Quiz Solution

Because a class is just an instance of Class, you can create it by calling Class.new. Class.new also accepts an argument (the superclass of the new class) and a block that is evaluated in the context of the newborn class:

```
c = Class.new(Array) do
  def my_method
    'Hello!'
  end
end
```

Now you have a variable that references a class, but the class is still anonymous. Do you remember the discussion about class names in *Constants*, on page 21? The name of a class is just a constant, so you can assign it yourself:

```
MyClass = c
```

Interestingly, Ruby is cheating a little here. When you assign an anonymous class to a constant, Ruby understands that you're trying to give a name to the class, and it does something special: it turns around to the class and says, "Here's your new name." Now the constant references the Class, and the Class also references the constant. If it weren't for this trick, a class wouldn't be able to know its own name, and you couldn't write this:

```
c.name    # => "MyClass"
```

You turn to Bill to show him your solution to the quiz—but he's already busy browsing the Bookworm source. It's time to get back to the task at hand.

Singleton Methods

Where it's your turn to teach Bill a few tricks.

It's late morning, and you and Bill are deep in the flow. You're zipping through the Bookworm source, deleting a useless line here, changing a confusing name there, and generally polishing the code…until you bump into a particularly troublesome bit of refactoring.

The Paragraph class wraps a string and then delegates all calls to the wrapped string—all of them, that is, except for one method, Paragraph#title?, which returns true if a Paragraph is all uppercase.

class_definitions/paragraph.rb

```
class Paragraph
  def initialize(text)
    @text = text
  end
```